

چرا Real-Time؟

از HTTP تا WebSocket

در Web معمولی، ارتباط Client و Server عمدتاً بر پایه‌ی مدل:

Client → Request → Server → Response → Client

انجام می‌شود.

مشکل زمانی ایجاد می‌شود که بخواهیم Server بتواند یک اتفاق جدید را بدون منتظر ماندن برای Request بعدی Client، به آن اطلاع دهد.

Short Polling

Client در فاصله‌های زمانی مشخص، مرتب از Server می‌پرسد:

"Anything new?"

مثلاً هر 3 ثانیه یک Request.

مشکل:

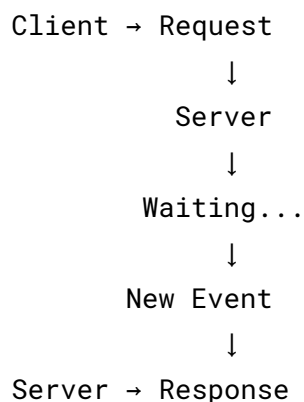
Fast polling → More requests

Slow polling → Higher latency

Long Polling

Client یک Request ارسال می‌کند و Server در صورت نبودن اطلاعات جدید، می‌تواند آن Request را مدتی باز نگه دارد.

وقتی اتفاق جدیدی رخ داد، Server Response را برمی‌گرداند.



Real-Time یعنی چه؟

در یک سیستم Real-Time، هدف این است که Client بتواند تغییرات جدید را با **تأخیر کم** دریافت کند، بدون اینکه برای هر تغییر نیاز باشد کاربر صفحه را Refresh کند یا خودش Request جدیدی برای همان تغییر ارسال کند.

مثال‌ها:

- Chat
- Notification
- Live Dashboard
- Monitoring
- Live Status Updates

WebSocket

WebSocket یک پروتکل جداگانه از HTTP برای ایجاد ارتباط **دوطرفه و مداوم** بین Client و Server است.

Client ⇄ Server

بعد از برقراری Connection، هر دو طرف می‌توانند مستقل از دیگری داده ارسال کنند.

HTTP و WebSocket

WebSocket خودش HTTP نیست.

برای شروع Connection، کلاینت، ابتدا یک HTTP Upgrade Request ارسال میکند:

HTTP Upgrade Request

↓

101 Switching Protocols

↓

WebSocket Connection

پس از Upgrade، ارتباط روی همان TCP Connection با WebSocket ادامه پیدا می‌کند.

SignalR

WebSocket یک راه برای ایجاد ارتباط Real-Time است.

اما برای ساخت یک Application واقعی، هنوز باید موارد زیادی را مدیریت کنیم.

SignalR در [ASP.NET Core](#) یک API سطح بالاتر برای ساخت ارتباطات Real-Time در اختیار ما قرار می‌دهد.

به‌صورت مفهومی:

Application

↓

SignalR

↓

Transport

↓

WebSocket / SSE / Long Polling

SignalR خودش WebSocket نیست.

SignalR می‌تواند از Transport‌های مختلف برای برقراری ارتباط استفاده کند.

- HTTP بر پایه‌ی Request / Response است.
- Short Polling مرتب Request ارسال می‌کند.
- Long Polling Request را تا رخ دادن یک Event نگه می‌دارد.
- Real-Time یعنی دریافت سریع تغییرات بدون نیاز به Request جدید برای هر تغییر.
- WebSocket یک پروتکل جداگانه برای ارتباط دوطرفه و مداوم است.
- SignalR ساخت ارتباطات Real-Time را در [ASP.NET Core](#) ساده‌تر می‌کند.